

Laboratoire 2

Roofline

Départements : TIC
Unité d'enseignement HPC

Auteurs : Urs Behrmann
Professeur : Alberto Dassatti
Assistant : Bruno Da Rocha Carvalho
Classe : HPC
Salle de labo : A09
Date : 23.03.2026

Contents

1	Roofline de la machine	3
1.1	Architecture CPU	3
1.2	Plafonds mesurés	3
1.3	Graphique Roofline	4
2	Mesures des cas d'étude	5
3	Analyse comparative	6
3.1	stream vs stride — memory coalescing	6
3.2	stream vs compute — mémoire vs calcul	6
3.3	rowmajor vs colmajor — localité spatiale et coalescing	6
4	Propositions d'amélioration théoriques	8
4.1	hpc_stride — tuilage (blocking)	8
4.2	hpc_compute — déroulage de boucle	8
4.3	colmajor — tuilage 2D	8

1 Roofline de la machine

1.1 Architecture CPU

Les mesures ont été faites sur un **AMD Ryzen 7 5800H** (architecture Zen3), avec 8 cœurs physiques (16 threads) et 16 MB de cache L3. Tout est en mono-thread (core 2), en double précision, compilé sans optimisation (-O0).

1.2 Plafonds mesurés

On a mesuré les deux paramètres du Roofline avec `likwid-bench` :

```
$ likwid-bench -t peakflops -W N:30kB:1
```

```
...
```

```
MFlops/s:    10712.35
```

```
MByte/s:     5356.17
```

```
$ likwid-bench -t load -W N:2GB:1
```

```
...
```

```
MByte/s:     26040.92
```

Paramètre	Valeur	Commande
Calcul max (<i>maxperf</i>)	10 712.35 MFlops/s	<code>likwid-bench -t peakflops -W N:30kB:1</code>
Bande passante max (<i>maxband</i>)	26 040.92 MB/s	<code>likwid-bench -t load -W N:2GB:1</code>

Le *knee* (point de transition) est à $AI = \frac{10712.35}{26040.92} \approx \mathbf{0.411}$ FLOP/Byte. En dessous de cette valeur le kernel est limité par la mémoire, au-dessus il est limité par le calcul.

1.3 Graphique Roofline

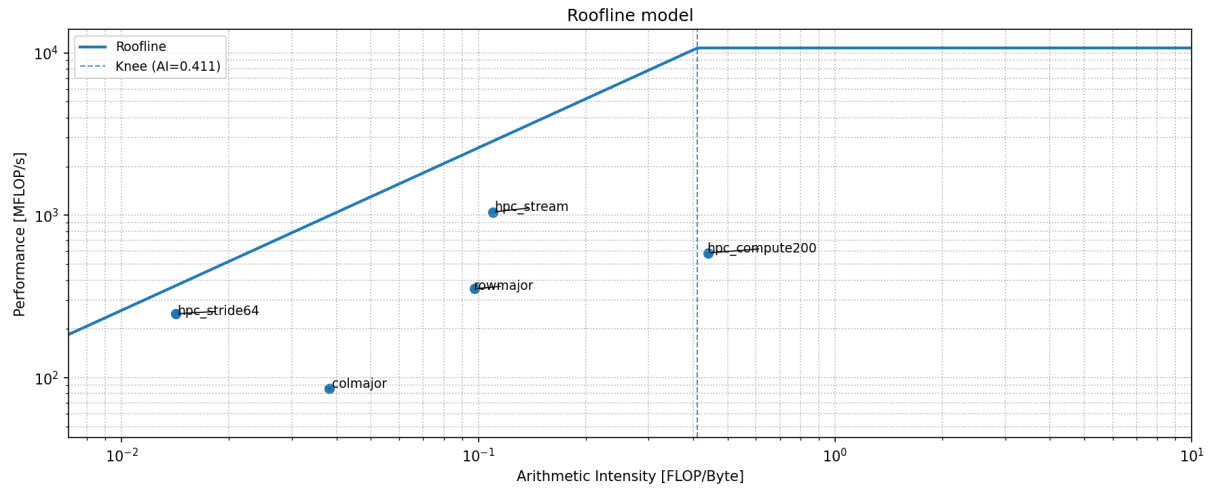


Figure 1: Roofline — AMD Ryzen 7 5800H, mono-thread, double précision scalaire, -O0. Tous les points sont sous le plafond théorique.

2 Mesures des cas d'étude

Les mesures ont été obtenues avec `likwid-perfctr` (groupe HPC, core 2, `sudo`) en utilisant les marqueurs LIKWID intégrés dans le code. Les valeurs AI et Perf proviennent directement des compteurs matériels du processeur.

```
sudo likwid-perfctr -C 2 -g HPC -m ./roofline_demo hpc_stream 50000000
```

Cas d'étude	AI [FLOP/Byte]	Perf [MFlops/s]	Paramètres
<code>hpc_stream</code>	0.1099	1048	<code>n=50 000 000</code>
<code>hpc_stride64</code>	0.0142	248	<code>n=50M, stride=64</code>
<code>hpc_compute200</code>	0.4406	586	<code>n=1M, iters=200</code>
<code>rowmajor</code>	0.0976	353	<code>N=4096</code>
<code>colmajor</code>	0.0382	86	<code>N=4096</code>

Table 1: Résultats des 4 cas d'étude — mesurés par compteurs matériels LIKWID.

Écart entre AI théorique et AI mesurée.

L'AI analytique suppose que seules les données utiles transitent par la DRAM. En réalité, plusieurs effets matériels augmentent le trafic réel :

- *Write-allocate* : avant d'écrire une ligne de cache, le processeur la charge d'abord depuis la DRAM, ce qui double le trafic pour les écritures.
- *Granularité des lignes de cache* : avec `hpc_stride64`, chaque accès charge 64 octets mais n'en utilise que 8 (12.5% d'efficacité), ce qui fait exploser le trafic DRAM réel et effondre l'AI mesurée de 0.188 à 0.014.
- *Réutilisation du cache* : pour `hpc_compute200`, les données tiennent en cache après le premier chargement. Le trafic DRAM est donc minime, ce qui donne une AI mesurée de 0.44 — très différente de l'AI théorique de 75.25.

3 Analyse comparative

3.1 stream vs stride — memory coalescing

`hpc_stream` et `hpc_stride64` ont la même AI théorique, mais les compteurs LIKWID révèlent une réalité très différente : AI=0.110 et 1048 MFlops/s pour `stream`, contre AI=0.014 et 248 MFlops/s pour `stride64`.

C'est un problème de **memory coalescing** (concept vu en CNM). Une ligne de cache fait 64 octets, soit 8 **double**. Quand les accès sont contigus (`stream`, `stride=1`), chaque ligne chargée est utilisée à 100% : on profite pleinement des 8 valeurs ramenées depuis la DRAM. C'est du bon coalescing.

Avec un stride de 64, chaque itération saute directement à la prochaine ligne de cache : on charge 64 octets mais on n'utilise qu'un seul **double** (8 octets), soit un rendement de 12.5%. LIKWID mesure 10.5 GB de trafic DRAM pour seulement 150M FLOPs — le trafic mémoire est 8× ce que l'analyse théorique prédit. C'est exactement ce qu'on cherche à éviter avec le coalescing : regrouper les accès pour charger chaque ligne une seule fois et l'utiliser entièrement.

3.2 stream vs compute — mémoire vs calcul

`hpc_stream` (AI = 0.110 mesuré, sous le *knee* à 0.411) est limité par la bande passante mémoire. Sa performance de 1048 MFlops/s est bien en dessous du plafond de calcul (10 712 MFlops/s), mais c'est normal : il n'y a pas assez d'opérations par octet transféré pour occuper le processeur.

`hpc_compute200` (AI = 0.44 mesuré) est un cas très intéressant : son AI se situe juste au-dessus du *knee* (0.411), donc à la frontière entre mémoire et calcul. Sa performance de 586 MFlops/s reste loin du plafond de 10 712 MFlops/s à cause du `-00` : sans vectorisation, le processeur ne fait qu'une opération scalaire à la fois alors qu'il pourrait en faire 4 en parallèle avec AVX2. L'AI théorique était 75.25, mais en pratique les données tiennent en cache — le vrai goulot d'étranglement est le calcul scalaire, pas la DRAM.

3.3 rowmajor vs colmajor — localité spatiale et coalescing

Ces deux kernels ont la même AI théorique, mais LIKWID révèle des valeurs très différentes : `rowmajor` (AI=0.098, 353 MFlops/s) est 4.1× plus rapide que `colmajor` (AI=0.038, 86 MFlops/s).

En C, les tableaux 2D sont stockés ligne par ligne (*row-major*). `rowmajor` parcourt les éléments dans l'ordre où ils sont en mémoire, donc les accès sont coalescés : chaque ligne

de cache chargée contient les 8 prochains éléments à utiliser.

`colmajor` fait le contraire : il saute de ligne en ligne, soit 4096 éléments ($4096 \times 8 = 32$ kB) entre deux accès consécutifs. Résultat : quasiment aucun réemploi de ligne de cache, les accès ne sont pas coalescés, et on se retrouve à faire autant de cache-misses que d'éléments lus.

4 Propositions d'amélioration théoriques

4.1 `hpc_stride` — tuilage (blocking)

Avec `stride=64`, chaque accès tombe dans une nouvelle ligne de cache et les 7 autres `double` chargés avec elle ne sont jamais utilisés.

L'idée du *tuilage* est de découper le tableau en petits blocs de taille B qui tiennent entièrement dans le cache L1 (32 kB sur Zen3). On traite un bloc entier avant de passer au suivant, ce qui force la réutilisation de chaque ligne de cache chargée. Au lieu d'accéder à `x[0]`, `x[64]`, `x[128]`..., on traiterait d'abord tous les éléments du bloc `x[0..B]`, puis `x[B..2B]`, etc.

4.2 `hpc_compute` — déroulage de boucle

Dans `heavy_work`, chaque itération dépend du résultat de la précédente (*dépendance de données*). Le processeur doit donc attendre la fin de chaque itération avant de commencer la suivante, même s'il pourrait en traiter plusieurs à la fois.

En déroulant la boucle (*loop unrolling*), on traite 4 éléments de `x` à la fois, chacun avec ses propres variables de calcul. Ces 4 chaînes sont indépendantes entre elles, donc le processeur peut les exécuter en parallèle et mieux occuper ses unités arithmétiques.

4.3 `colmajor` — tuilage 2D

L'accès colonne par colonne saute de $N \times 8 = 32$ kB entre deux itérations consécutives (pour $N=4096$), soit 512 lignes de cache. Chaque accès est un cache-miss.

Avec un tuilage $B \times B$ (avec B choisi pour que le bloc tienne dans le cache L1), on réorganise les boucles pour travailler sur un petit bloc à la fois. Les lignes de cache chargées pour les premières itérations du bloc sont encore en cache pour les suivantes, ce qui réduit drastiquement le nombre de cache-misses.